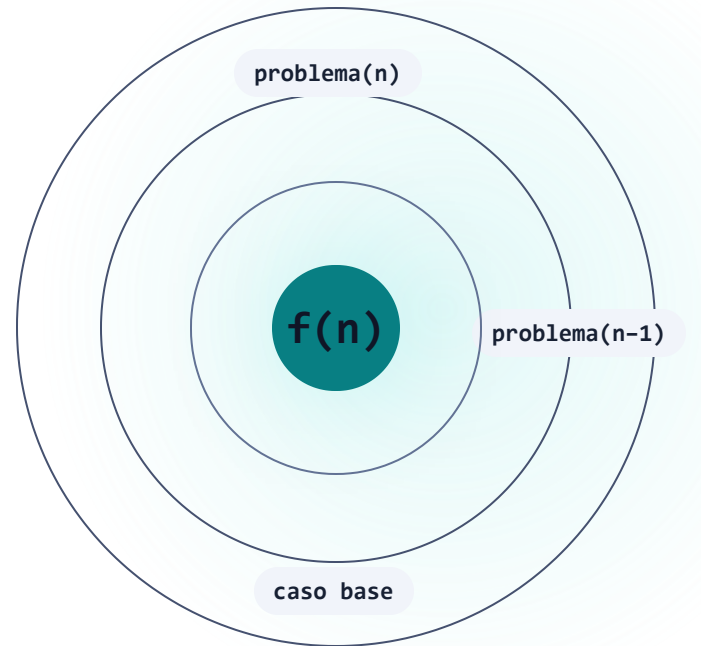


Resolver lo grande pensando en pequeño.

Aprende cómo una función resuelve un problema llamándose a sí misma, qué relación tiene con la pila de ejecución y dónde aparece en listas y árboles. Sigue los ejemplos introductorios y estudia su implementación en Java.



¿Qué es la recursividad?

En programación, una función o método es recursivo cuando, durante su ejecución, vuelve a invocarse para resolver una versión más pequeña del mismo problema. Es una forma de diseñar algoritmos, no una estructura de datos por sí misma [1], [2].

1

Caso base

Es la respuesta directa que detiene las llamadas. Puede haber más de un caso base. Sin uno alcanzable, el método continúa hasta causar un desbordamiento de pila.

2

Caso recursivo

Resuelve una parte del problema y llama al método con datos más pequeños. Cada paso debe acercarse al caso base.

3

Retorno

Cuando termina el caso base, cada llamada pendiente recibe un resultado y completa su operación.

1. Recibir datos



2. Comprobar caso base



3. Reducir el problema



4. Llamarse de nuevo



5. Retornar resultado

Recursión directa e indirecta

Directa: un método se llama a sí mismo.

Indirecta o mutua: un método A llama a B y B vuelve a llamar a A. En los dos casos, el ciclo necesita una condición que lo termine.

¿Cómo comprobar que termina?

Identifica una medida que disminuya en cada llamada —por ejemplo, un índice que avanza hacia el final de una lista— y verifica que llegue al caso base. Si la entrada no se reduce, la recursión puede no terminar.

Cuándo conviene. La recursividad se adapta bien a problemas definidos por versiones más pequeñas de sí mismos, como recorrer árboles, explorar laberintos o dividir un problema. Puede hacer el código más claro, pero cada llamada consume espacio en la pila; por eso conviene comparar claridad, memoria y tiempo con una solución iterativa [2], [3].

Sobre los ejemplos: los algoritmos que se explican y se pueden copiar están escritos en Java. Los controles del sitio usan JavaScript únicamente para actualizar los dibujos y resultados interactivos en el navegador.

Recursividad y estructuras de datos

En Estructura de Datos, la recursividad es especialmente útil cuando la forma de los datos contiene partes del mismo tipo. También permite observar cómo el programa administra las llamadas pendientes.

A

Pila de llamadas

Java guarda cada llamada activa en la pila de ejecución. Su marco conserva parámetros, variables locales y el punto al que debe volver. Las llamadas se retiran en orden LIFO: la última en entrar es la primera en terminar.

B

Listas enlazadas

Cada nodo apunta al siguiente nodo. Un recorrido recursivo procesa el nodo actual y continúa desde `nodo.siguiente`; el caso base ocurre al llegar a `null`.

C

Árboles

Cada nodo puede tener subárboles, que también son árboles. Por eso operaciones como contar nodos, buscar un valor y recorrer en preorden, inorden o postorden se expresan naturalmente con recursión.

La pila durante factorial(3)

fondo → factorial(3)

factorial(2)

factorial(1)

cima → factorial(0)

Al resolver factorial(0), Java regresa por la pila: $1 \rightarrow 1 \times 1 \rightarrow 2 \times 1 \rightarrow 3 \times 2 = 6$. Cada marco queda pendiente hasta recibir el valor de la llamada siguiente.

Recorrer una lista enlazada en Java

```
public class RecorrerLista {
    static class Nodo {
        int valor;
        Nodo siguiente;
        Nodo(int valor) {
            this.valor = valor; }

        static void imprimir(Nodo
nodo) {
            if (nodo == null) return;
            // caso base

            System.out.println(nodo.valor);
            imprimir(nodo.siguiente);
        }

        public static void
main(String[] args) {
            Nodo primero = new
Nodo(10);
            primero.siguiente = new
Nodo(20);

            primero.siguiente.siguiente = new
Nodo(30);
            imprimir(primero);
        }
    }
}
```

Guarda como RecorrerLista.java. El parámetro avanza al nodo siguiente hasta llegar a null; el programa imprime 10, 20 y 30. Para n

nodos, recorre cada nodo una vez: $\Theta(n)$ tiempo y $\Theta(n)$ espacio de pila.

Recorrer un árbol binario en Java

```
public class RecorrerArbol {
    static class Nodo {
        int dato;
        Nodo izquierda, derecha;
        Nodo(int dato) { this.dato
= dato; }
    }

    static void preorden(Nodo
nodo) {
        if (nodo == null) return;
        // caso base

        System.out.println(nodo.dato); //
raíz

        preorden(nodo.izquierda);
        // subárbol izq.
        preorden(nodo.derecha);
        // subárbol der.
    }

    public static void
main(String[] args) {
        Nodo raiz = new Nodo(1);
        raiz.izquierda = new
Nodo(2);
        raiz.derecha = new
Nodo(3);
        preorden(raiz); // imprime
1, 2, 3
    }
}
```

Guarda como `RecorrerArbol.java`. El recorrido `preorden` visita primero la raíz, luego el subárbol izquierdo y por último el derecho. Al llegar a un

Leer el orden del recorrido

1 • visitar raíz

2 • recorrer izquierda

3 • recorrer derecha

4 • regresar al padre

El mismo método se aplica en cada subárbol. Cuando un subárbol termina, la llamada regresa a su padre y continúa con la rama pendiente. Para un árbol con n nodos, `preorden` tarda $\Theta(n)$; la pila usa espacio proporcional a la altura del árbol.

enlace vacío (null), el caso base detiene esa rama.

Regla práctica: antes de escribir un método recursivo, define qué significa su parámetro, dónde termina, cómo se acerca a la terminación y qué resultado devuelve cada llamada.

03 — ANÁLISIS

¿Cuánto cuesta recursar?

El análisis suele expresar el trabajo como una recurrencia: el costo de una instancia depende del costo de instancias más pequeñas [2], [3]. También hay que contar la memoria de las llamadas pendientes en la pila.

Algoritmo	Tiempo típico	Memoria de pila	Observación
Factorial recursivo	$\Theta(n)$	$\Theta(n)$	Una llamada por nivel.
Fibonacci ingenuo	$\Theta(\phi^n)$	$\Theta(n)$	$\phi \approx 1.618$; recalcula subproblemas.
Fibonacci con memoización	$\Theta(n)$	$\Theta(n)$	Guarda cada resultado para reutilizarlo.
Torres de Hanói	$\Theta(2^n)$	$\Theta(n)$	La solución mínima requiere $2^n - 1$ movimientos.
Triángulo de Sierpiński	$\Theta(3^d)$	$\Theta(d)$	d es la profundidad; cada nivel dibuja 3 copias.

Las cotas describen estas implementaciones y modelos de costo; optimizaciones, representación de números y límites de la plataforma pueden cambiar el costo observado.

Factorial: una cadena de dependencias

Para $n \geq 1$, $n! = n \times (n - 1)!$. El caso base es $0! = 1$. La función reduce n hasta llegar a cero y luego multiplica los resultados al regresar.

Prueba el cálculo

n (0 a 18)

5! = 120

Bajada y retorno de la recursión:



Programa de ejemplo en Java

```

public class Factorial {
    static long factorial(int n)
    {
        if (n < 0 || n > 20) {
            throw new
IllegalArgumentException("Usa n
entre 0 y 20");
        }
        if (n == 0) return 1; //
caso base
        return n * factorial(n -
1); // caso recursivo
    }

    public static void
main(String[] args) {

        System.out.println(factorial(5));
        // imprime 120
    }
}
  
```

Guarda como `Factorial.java`. Traza:
`factorial(5)` espera a `factorial(4)`, que espera a `factorial(3)`, hasta `factorial(0) = 1`. Al regresar: $1 \times 2 \times 3 \times 4 \times 5 = 120$. `long` admite factorial hasta 20 sin desbordarse.

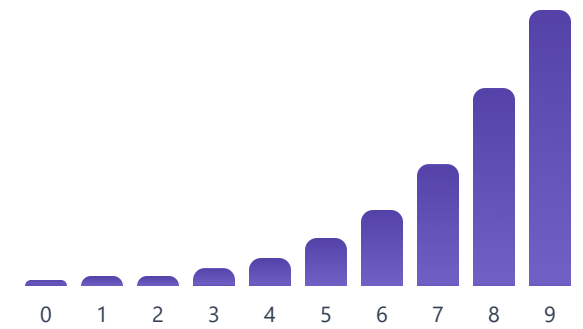
Fibonacci: dos llamadas por nivel

Cada término es la suma de los dos anteriores: $F(0) = 0$, $F(1) = 1$ y $F(n) = F(n - 1) + F(n - 2)$ [4]. Esta versión introductoria ilustra la recursividad, pero repite cálculos; memoizar evita repetirlos.

Construye la secuencia

Cantidad de términos: **10**

0, 1, 1, 2, 3, 5, 8, 13, 21, 34



Esta gráfica usa memoización para responder rápidamente; el programa Java recursivo básico aparece al lado.

Programa recursivo en Java

```
public class Fibonacci {
    static long fibonacci(int n)
    {
        if (n < 0 || n > 30) {
            throw new
IllegalArgumentException("Usa n
entre 0 y 30");
        }
        if (n <= 1) return n; //
casos base
        return fibonacci(n - 1) +
fibonacci(n - 2);
    }

    public static void
main(String[] args) {

System.out.println(fibonacci(7));
// imprime 13
    }
}
```

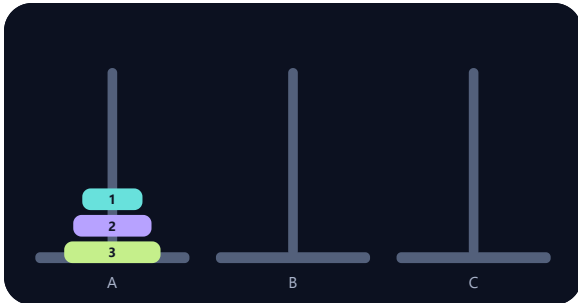
Guarda como `Fibonacci.java`. Para $n = 7$, la respuesta es $F(6) + F(5) = 8 + 5 = 13$. El límite 30 evita que el ejemplo introductorio tarde demasiado: la versión básica recalcula términos y su tiempo crece exponencialmente. Después se puede optimizar con memoización [4].

Torres de Hanói: mover para resolver

Mueve una torre de discos entre tres postes: solo se mueve un disco a la vez y nunca se coloca uno grande sobre uno pequeño. Para n discos, mueve $n - 1$ al poste auxiliar, mueve el mayor y vuelve a mover $n - 1$ [5].

Observa la solución

Discos: 3



Movimiento 0 de 7 · posición inicial.

Programa recursivo en Java

```
public class TorresDeHanoi {
    static void mover(int n, char
                        origen,
                        char
                        destino, char auxiliar) {
        if (n == 0) return; //
    caso base
        mover(n - 1, origen,
            auxiliar, destino);
        System.out.println(origen
            + " -> " + destino);
        mover(n - 1, auxiliar,
            destino, origen);
    }

    public static void
    main(String[] args) {
        mover(3, 'A', 'C', 'B');
    }
}
```

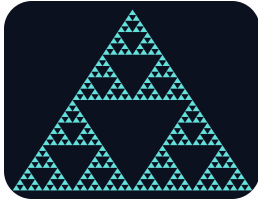
Guarda como `TorresDeHanoi.java`. Cada línea impresa es un movimiento legal. La regla es: mover $n - 1$ discos al auxiliar, mover el disco mayor y mover los $n - 1$ al destino. Para 3 discos se imprimen 7 movimientos; en general, $2^n - 1$ [5].

Fractales: patrones que se repiten

Un fractal puede construirse repitiendo una regla geométrica a distintas escalas. El triángulo de Sierpiński se genera dividiendo un triángulo en cuatro y repitiendo el proceso en los tres triángulos de las esquinas [6].

Triángulo de Sierpiński

Profundidad: **5**



243 triángulos
mínimos en
profundidad 5.

Programa gráfico con Java Swing

```
import java.awt.*;
import java.awt.geom.Path2D;
import java.awt.geom.Point2D;
import javax.swing.*;

public class Sierpinski extends JPanel {
    @Override
    protected void paintComponent(Graphics graphics) {
        super.paintComponent(graphics);
        Graphics2D g = (Graphics2D) graphics.create();
        g.setColor(new Color(30, 145, 150));
        sierpinski(g, new Point2D.Double(360, 20),
            new Point2D.Double(20, 540),
            new Point2D.Double(700, 540), 5);
        g.dispose();
    }

    static void sierpinski(Graphics2D g,
        Point2D.Double a, Point2D.Double b,
        Point2D.Double c, int nivel) {
        if (nivel == 0) {
            Path2D figura = new Path2D.Double();
            figura.moveTo(a.x, a.y);
            figura.lineTo(b.x, b.y);
            figura.lineTo(c.x, c.y);
            figura.closePath();
            g.fill(figura);
            return;
        }
        Point2D.Double ab = medio(a, b);
        Point2D.Double bc = medio(b, c);
        Point2D.Double ca = medio(c, a);
        sierpinski(g, a, ab, ca, nivel - 1);
        sierpinski(g, ab, b, bc, nivel - 1);
        sierpinski(g, ca, bc, c, nivel - 1);
    }
}
```

```

static Point2D.Double medio(Point2D.Double p,
                             Point2D.Double q) {
    return new Point2D.Double((p.x + q.x) / 2,
                              (p.y + q.y) / 2);
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        JFrame ventana = new JFrame("Triángulo de
Sierpiński");

        ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        ventana.setContentPane(new Sierpinski());
        ventana.setSize(720, 600);
        ventana.setVisible(true);
    });
}

```

Guarda como `Sierpinski.java` y ejecútalo con `javac Sierpinski.java` y `java Sierpinski`. En cada nivel dibuja tres subtriángulos; por eso la cantidad crece como 3^d . La visualización interactiva de esta página dibuja el mismo patrón en el navegador.

Una idea pequeña, muchas soluciones

En Estructura de Datos, la recursividad conecta la definición de un algoritmo con la forma de los datos que procesa. Listas y árboles se pueden recorrer siguiendo sus enlaces o subárboles; la pila de ejecución explica cómo se guardan las llamadas que faltan por terminar. Al dominar los casos base, la reducción, el retorno y el análisis de tiempo y espacio, podemos escribir recorridos correctos y entender cuándo una solución recursiva es más clara —y cuándo conviene reemplazarla por una iterativa para controlar la memoria.

Equipo de trabajo — exactamente 5 integrantes

Completen los nombres antes de publicar el sitio; no agreguen ni eliminen espacios para mantener el equipo de cinco personas solicitado.

Integrante 1

Oswaldo Florencio Mateo.

Integrante 2

Alexis Martínez Flores

Integrante 3

Marbella Martínez Luis

Integrante 4

Omar Mendoza Nicolás

Integrante 5

Moises Reyes Dolores

Referencias

Fuentes académicas y técnicas consultadas para las definiciones, recurrencias, estructuras y análisis. Se utilizaron seis fuentes.

1. [1] P. E. Black, "recursion," en *Dictionary of Algorithms and Data Structures*, National Institute of Standards and Technology (NIST), 21 abr. 2022. [En línea]. Disponible: <https://xlinux.nist.gov/dads/HTML/recursion.html>. [Accedido: 7-oct-2026].
2. [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4.^a ed. Cambridge, MA, EE. UU.: MIT Press, 2022.
3. [3] Algorithms and Theory of Computation Handbook, "recurrence relation," en *Dictionary of Algorithms and Data Structures*, NIST, 17 dic. 2004. [En línea]. Disponible: <https://xlinux.nist.gov/dads/HTML/recurrence.html>. [Accedido: 7-oct-2026].
4. [4] P. Rodgers, "Fibonacci number," en *Dictionary of Algorithms and Data Structures*, NIST, 21 abr. 2022. [En línea]. Disponible: <https://xlinux.nist.gov/dads/HTML/fibonacciNumber.html>. [Accedido: 7-oct-2026].
5. [5] P. E. Black, "towers of Hanoi," en *Dictionary of Algorithms and Data Structures*, NIST, 21 abr. 2022. [En línea]. Disponible: <https://xlinux.nist.gov/dads/HTML/towersOfHanoi.html>. [Accedido: 7-oct-2026].
6. [6] E. W. Weisstein, "Sierpiński Triangle," *Wolfram MathWorld*. [En línea]. Disponible: <https://mathworld.wolfram.com/SierpinskiTriangle.html>. [Accedido: 7-oct-2026].